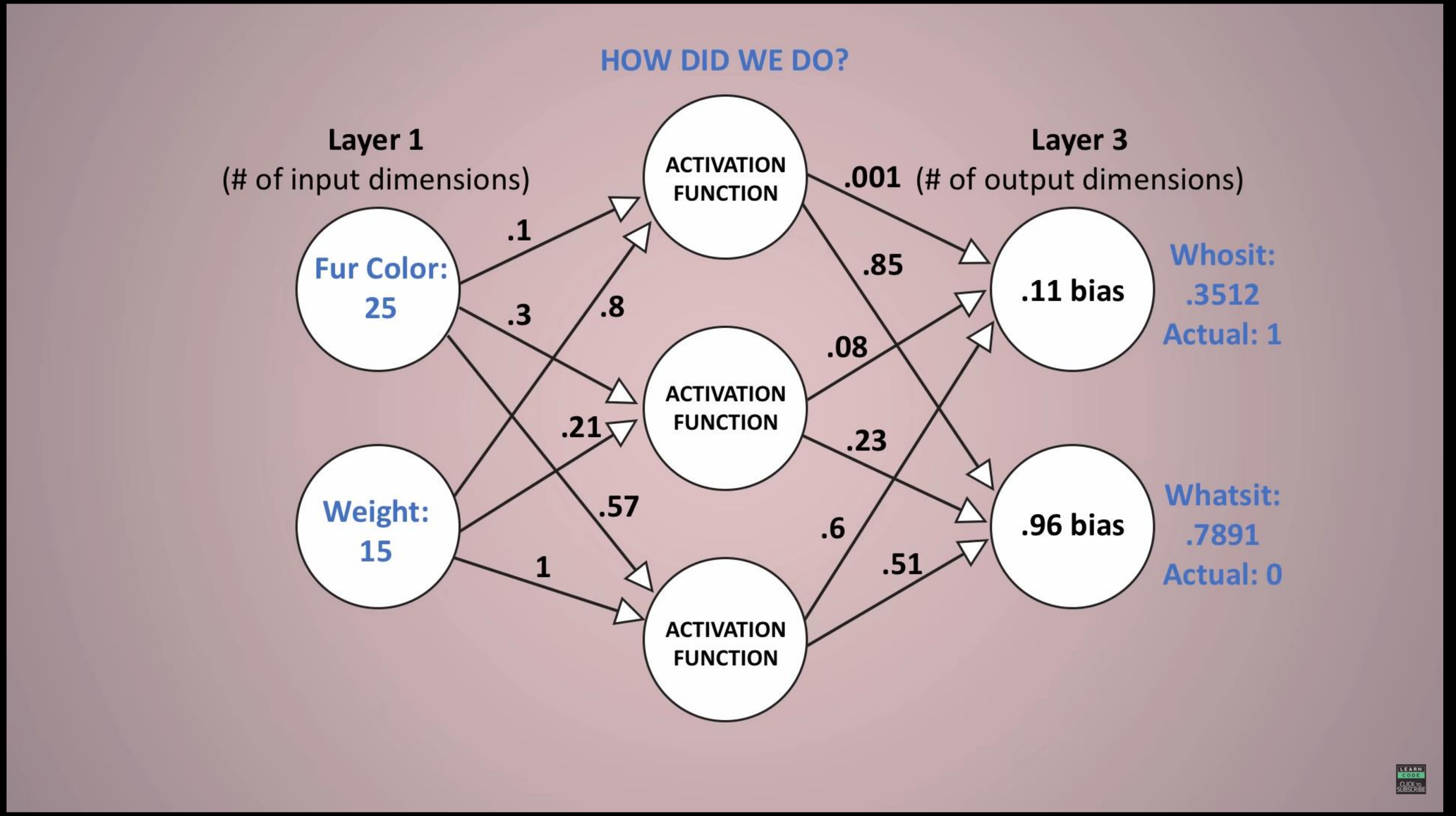
07.12.21

OPENCAMPUS.sh

Einführung in Data Science und maschinelles Lernen

BEHANDLUNG VON FEHLENDEN WERTEN

- Wiederholung wichtiger Konzepte des maschinellen Lernens
- Implementierung eines NN mit TensorFlow und Python
- Behandlung fehlender Werte
- (Support-Vektor-Maschinen)


HOW DID WE DO?
Layer 1
(# of input dimensions)
Layer 3
(# of output dimensions)
Fur Color:
25
Weight:
15
ACTIVATION
FUNCTION
ACTIVATION
FUNCTION
ACTIVATION
FUNCTION
.11 bias
.96 bias
.1
.3
.8
.21
.57
1
.001
.85
.08
.23
.6
.51
Whosit:
.3512
Actual: 1
Whatsit:
.7891
Actual: 0

NEURONALE NETZE

Input Layer

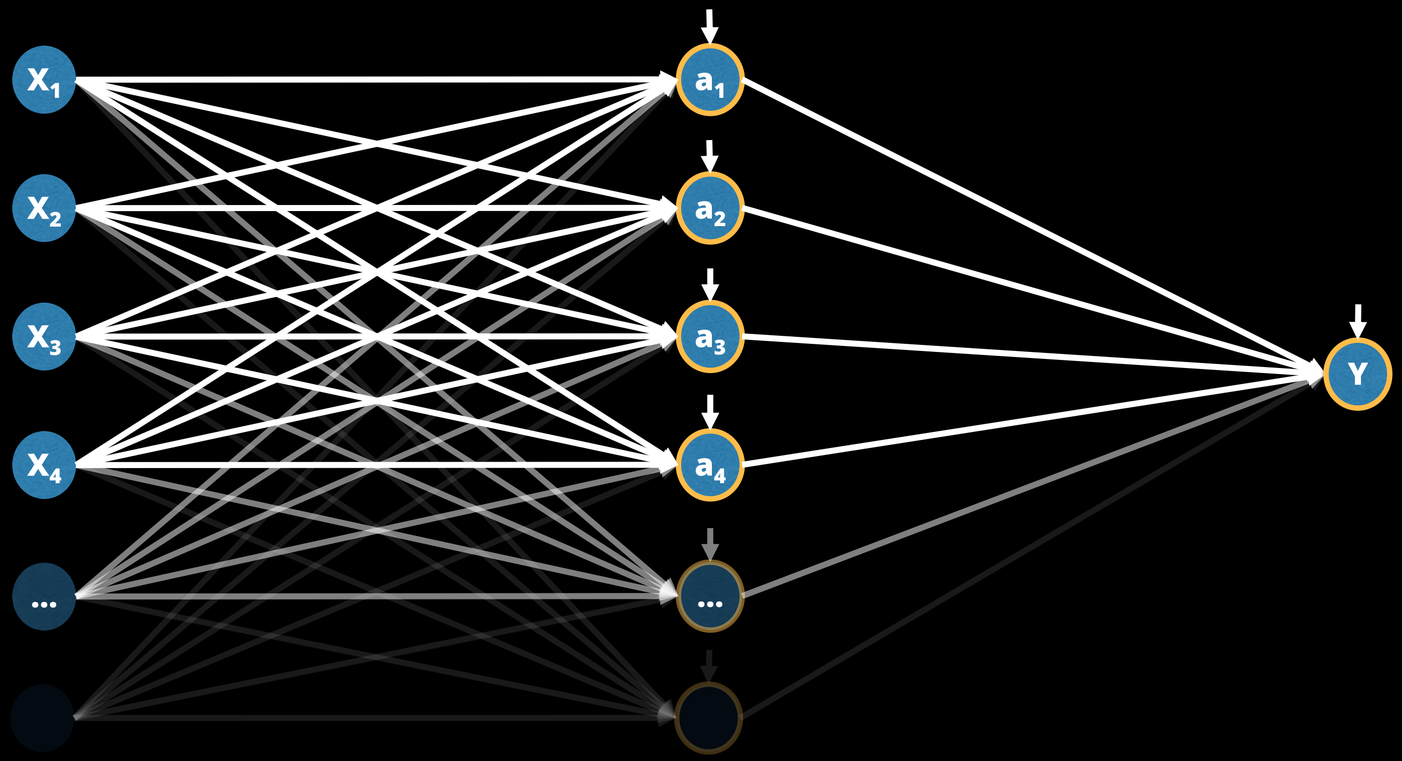
Besteht aus Input-Variablen/ Features/ Dimensionen

Hidden Layer

Fasst mit Hilfe von Aktivierungsfunktionen und geschätzten Gewichten die Werte der vorherigen Schicht in jeweils einem Neuron zusammen.

Output Layer

Fasst ebenfalls mit Hilfe von Aktivierungsfunktion und geschätzten Gewichten die Werte der vorherigen Schicht zusammen.



WICHTIGE KONZEPTE

Forward Propagation:

- **Berechnung der Vorhersage basierend auf den aktuellen Parametern und den definierten Aktivierungsfunktionen**

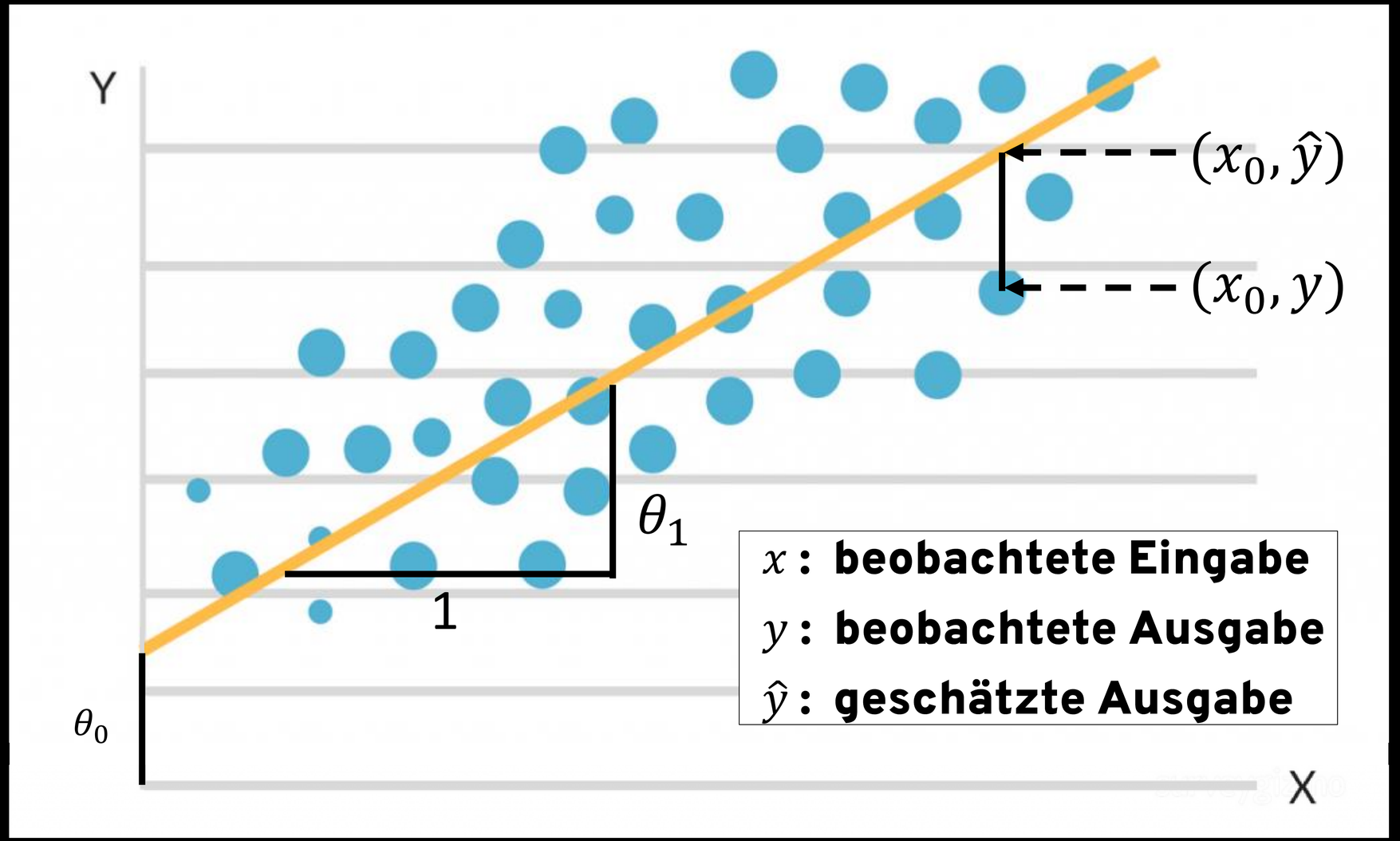
Backward Propagation:

- **Berechnung des Schätzfehlers mit Hilfe der Kostenfunktion**
- **Berechnung neuer, verbesserter Parameter mit Hilfe der Optimierungsfunktion**

LINEARES MODELL

$$\hat{y} = \theta_0 + \theta_1 x$$

$$= h_x\,(\theta_0, \theta_1)$$



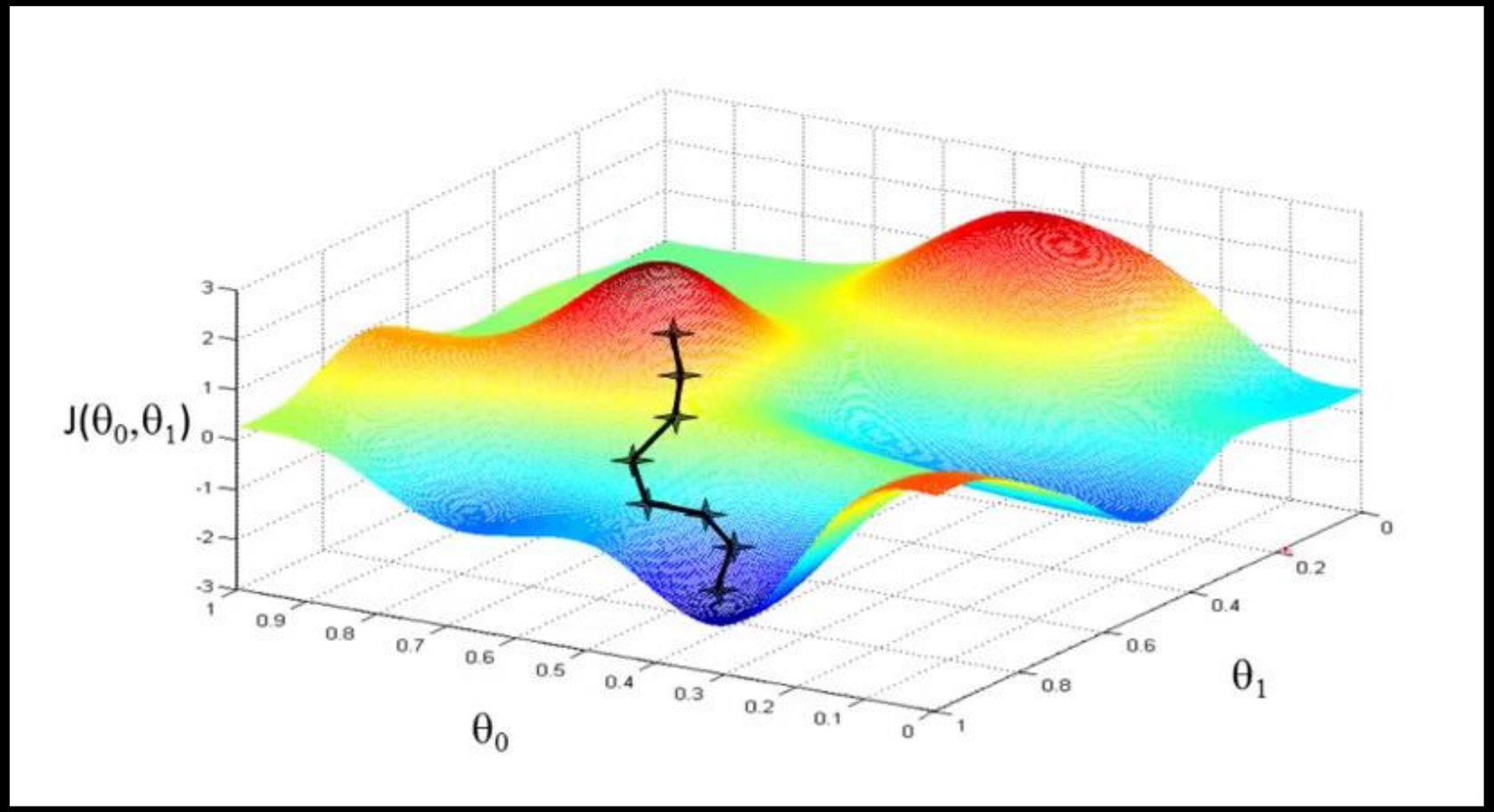
KOSTENFUNKTION

Zur Berechnung der Funktion mit den optimalen Parametern θ_0 und θ_1:

$$J_x(\theta_0, \theta_1) = \qquad (h_x(\theta_0, \theta_1) - y)$$

Mean Squared Error (MSE)

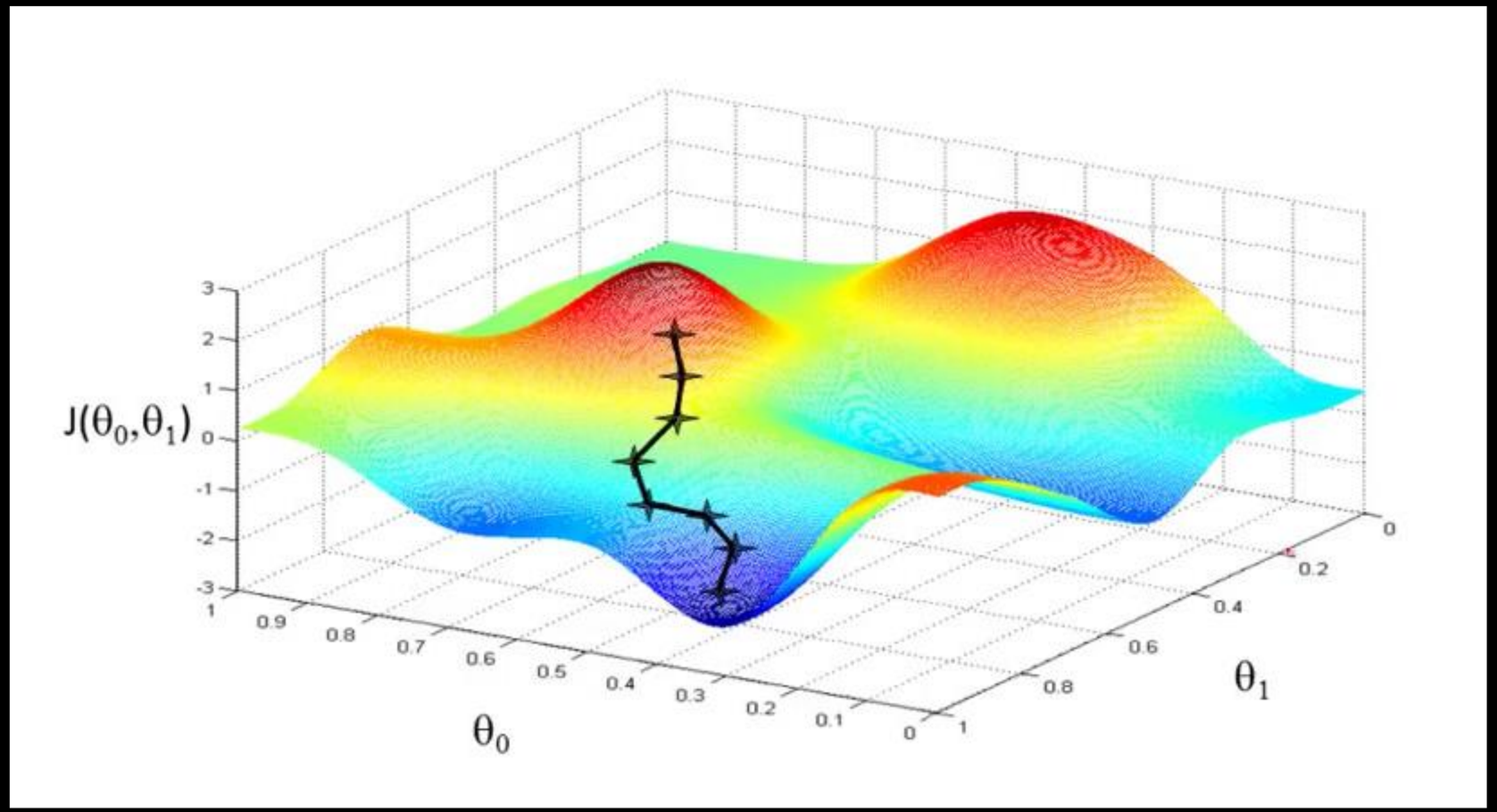
MINIMIERUNG DER KOSTENFUNKTION ÜBER EINE OPTIMIERUNGSFUNKTION



- **Minimierung der Kostenfunktion durch ein iteratives Verfahren (Gradient Descent), das sich dem Minimum annähert.**
- **Die Schrittgröße für die Annäherung kann durch die Lernrate („Learning Parameter“) kontrolliert werden.**

Quelle: https://www.coursera.org/learn/machine-learning

PARAMETER VON OPTIMIERUNGSFUNKTIONEN



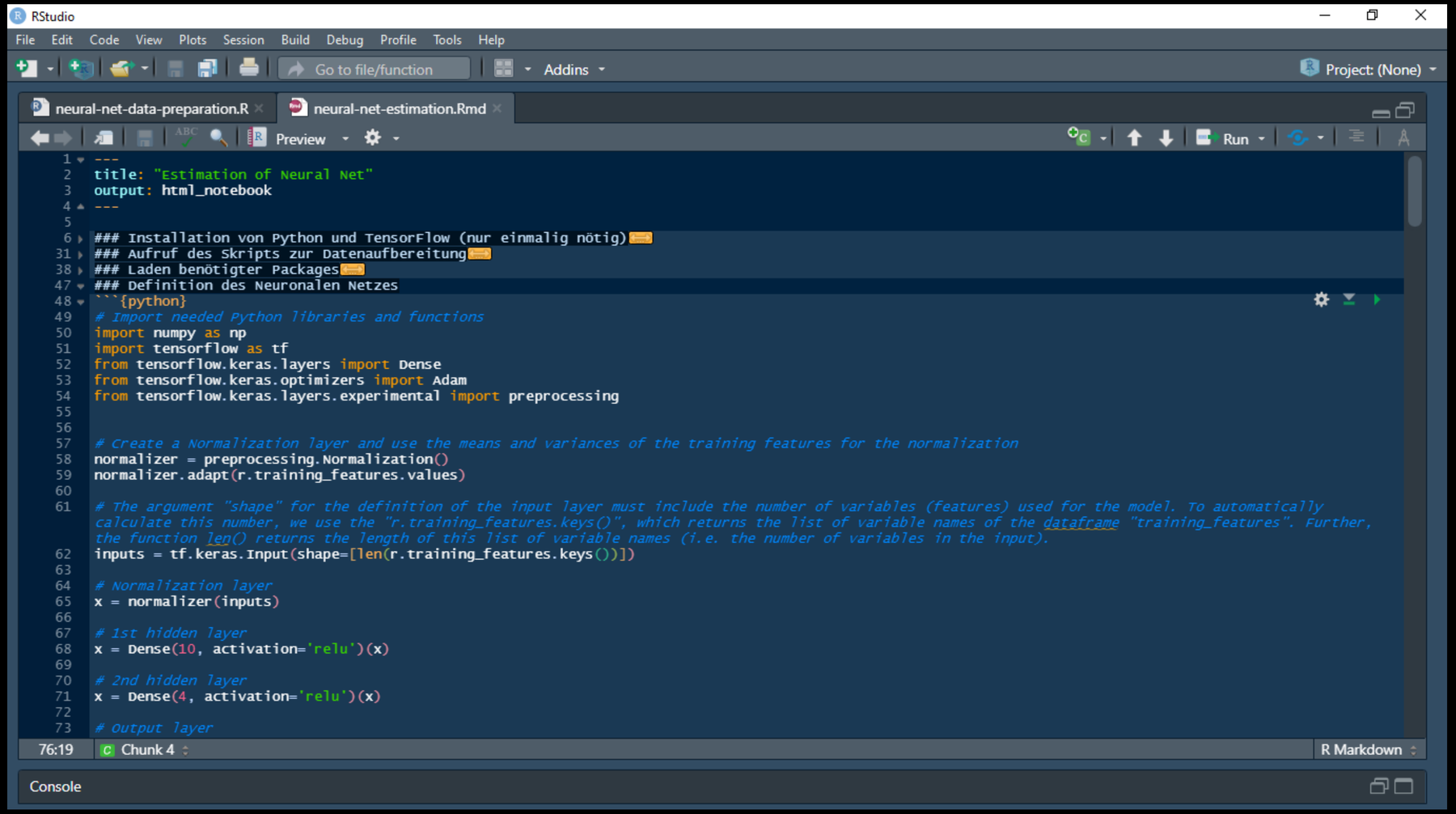
- **Schrittgröße für die Annäherung an das Kosten-Minimum („Learning Rate“)**
- **Trägheit bei Richtungsänderungen („Momentum“)**

Quelle: https://www.coursera.org/learn/machine-learning

DATENAUFBEREITUNG FÜR TENSORFLOW

```
1
2  ##################################################
3  ### Preparation of the Environment
20 ##################################################
21 ### Data Import
27 ##################################################
28 ### Data Preparation ####
29
30 # Recoding of the variables into one-hot encoded (dummy) variables
31 dummy_list <- c("view", "condition")
32 house_pricing_dummy = dummy_cols(house_pricing, dummy_list)
33
34 # Definition of lists for each one-hot encoded variable (just to make the handling easier)
35 condition_dummies = c('condition_1', 'condition_2', 'condition_3', 'condition_4', 'condition_5')
36 view_dummies = c('view_0', 'view_1', 'view_2', 'view_3','view_4')
37
38
39 ##################################################
40 ### Selection of the Feature Variables and the Label Variable ####
41
42 # Selection of the features (the independent variables used to predict the dependent)
43 #features <- c('sqft_lot', 'waterfront', 'grade', 'bathrooms', view_dummies, condition_dummies)
44 features <- c('sqft_lot', 'waterfront', 'grade', 'bathrooms', condition_dummies, view_dummies)
45 # Selection of the label (the dependent variable)
46 labels <- 'price'
47
48
49 ##################################################
50 ### Selection of Training, Validation and Test Data ####
51
52 # Look at the data
53 str(house_pricing_dummy)
54
55 # Setting the random counter to a fixed value, so the random initialization stays the same (the ra
56 set.seed(1)
57
```

DEFINITION EINES NEURONALEN NETZES



HYPERPARAMATER IN NEURONALEN NETZEN

- **Wahl der Architektur:**
  - **Anzahl der Hidden Layer des Netzes**
  - **Typen der Hidden Layer**
  - **Anzahl der Neuronen je Hidden Layer**
  - **Wahl der Aktivierungsfunktionen**
- **Wahl der Kostenfunktion („Loss Function“)**
- **Wahl der Optimierungsfunktion („Optimizer“)**
- **Wahl der Parameter des Optimizers**

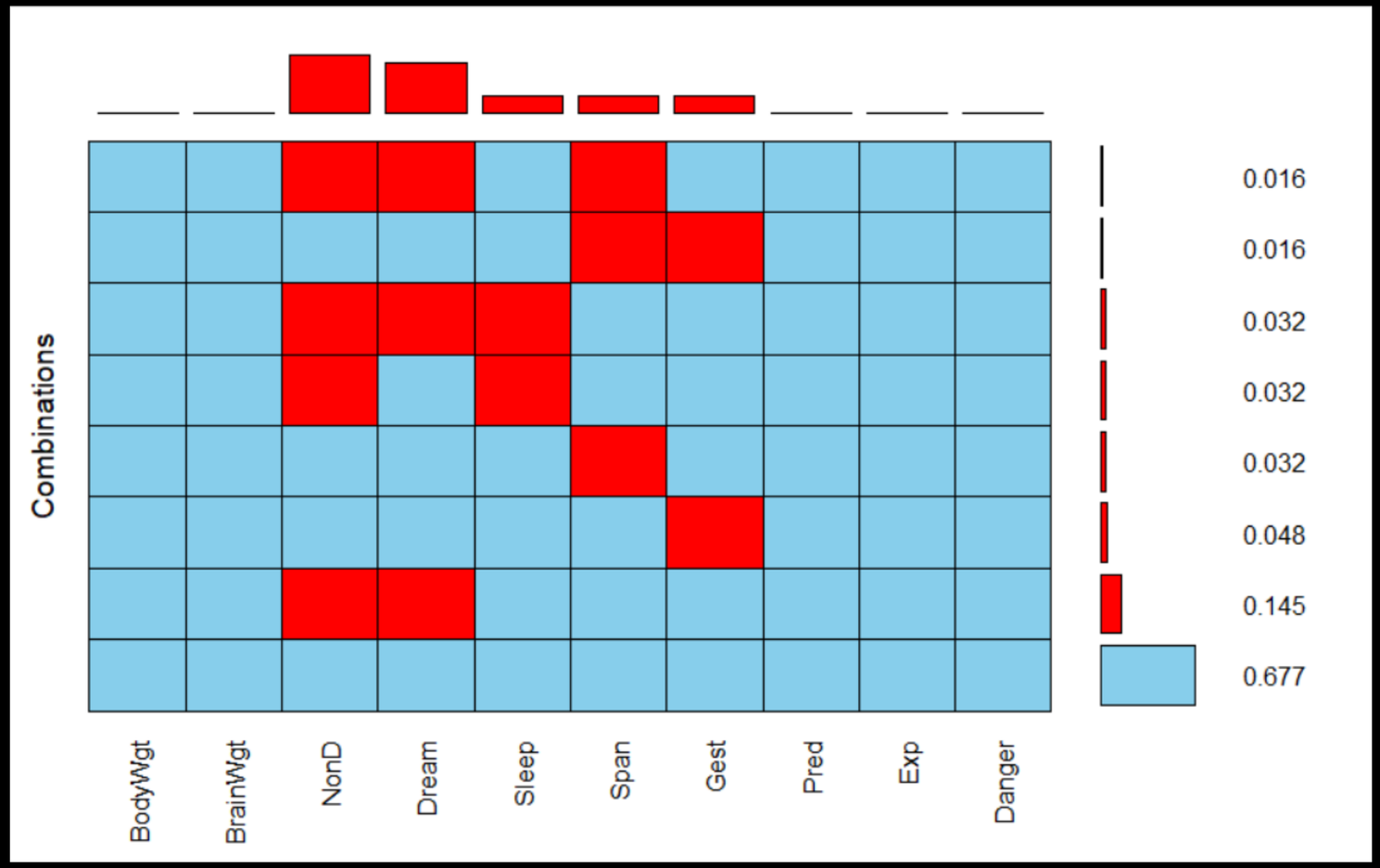
GRÜNDE FÜR FEHLENDE WERTE

- Nicht gegebene Antworten in Umfragen
- Zusammenführung von Daten aus verschiedenen Quellen mit unterschiedlichen Variablen
- Technische Probleme in der Datenerhebung oder Aufzeicnung
- …

TYPEN VON FEHLENDEN WERTEN

- **Missing Completely at Random (MCAR)**
- **Missing at Random (MAR)**
- **Missing not at Random (MNAR)**

VISUALISIERUNG FEHLENDER WERTE



VIM package (Visualization and Imputation of Missing Values)

```
aggr(dataset,
  combined=TRUE,
  numbers=TRUE)
```


Kowarik, A., & Templ, M. (2016). Imputation with the R Package VIM. *Journal of Statistical Software*, *74*, 1–16.

BEHANDLUNG VON FEHLENDEN WERTE

- Zugehörige Fälle löschen (listwise deletion)

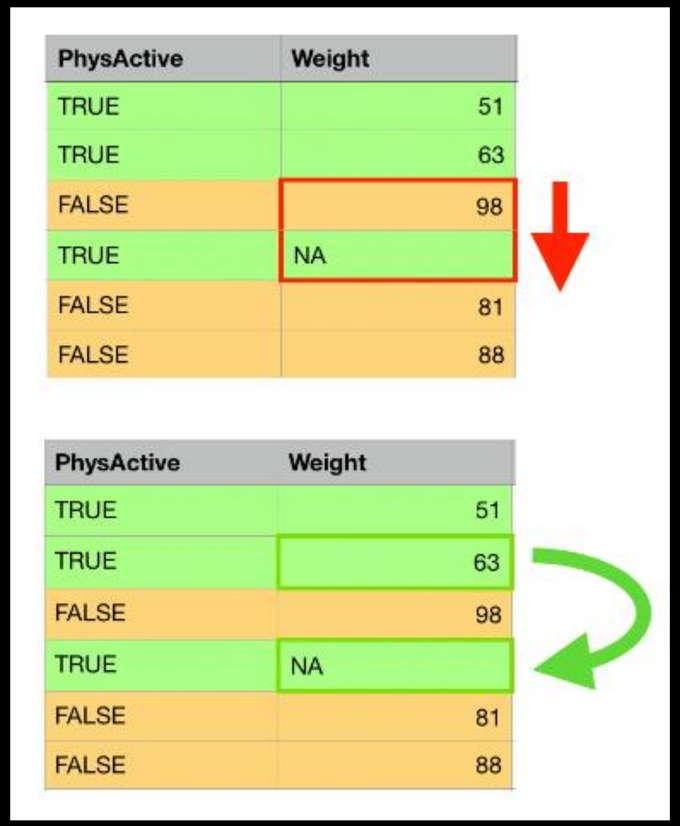
- Einfache „Spender“-basierte Imputation (donor-based)
  - Mittelwertschätzung (bzw. Median oder Mode)
  - nach „Ähnlichkeit“ (Hot-Deck Imputation)
  - durch minimalen Abstand (k Nearest Neighbors)

- Einfache modellbasierte Imputation
  - Iterative Regression

- Multiple Imputation

HOT-DECK IMPUTATION

Nach Domänen



PhysActive	Weight
TRUE	51
TRUE	63
FALSE	98
TRUE	NA
FALSE	81
FALSE	88

PhysActive	Weight
TRUE	51
TRUE	63
FALSE	98
TRUE	NA
FALSE	81
FALSE	88

Nach Korrelation



Height	Weight
150	51
161	63
189	98
155	NA
182	81
184	88

Height	Weight
150	51
155	NA
161	63
182	81
184	88
189	98

Oleszak, M. (2021). Hot-deck imputation | R. Retrieved December 6, 2021, from Datacamp website: https://campus.datacamp.com/courses/handling-missing-data-with-imputations-in-r/donor-based-imputation?ex=5

K NEAREST NEIGHBORS (KNN)

- Suche nach den k Fällen mit minimalem Abstand
  - je nach Variablentyp unterschiedliche Abstandsmessung
  - Zusammenführung der Abstände über eine Summenfunktion

- Zufällige Ziehung aus den k Fällen

ITERATIVE REGRESSION

A	B	C	D
5	34	NA	1
1	22	NA	4
NA	65	55	2
4	87	27	2
NA	23	10	1

ITERATIVE REGRESSION

1) Vorhersage fehlender Werte in A

A	B	C	D
5	34	**NA**	1
1	22	**NA**	4
5	65	55	2
4	87	27	2
2	23	10	1

ITERATIVE REGRESSION

2) Vorhersage Fehlender Werte in C mit den imputierten werten von A

A	B	C	D
5	34	**32**	1
1	22	**16**	4
5	65	55	2
4	87	27	2
2	23	10	1

ITERATIVE REGRESSION

3) Vorhersage fehlender Werte in A mit den imputierten Werten von C

A	B	C	D
5	34	**32**	1
1	22	**16**	4
NA	65	55	2
4	87	27	2
NA	23	10	1

→ Wiederholung bis keine Änderung mehr eintritt

ITERATIVE REGRESSION

1) Gehe schrittweise durch alle Variablen des Datensatz
2) Stelle dabei für jede Variable eine Regressionsmodell basierend auf allen anderen Variablen auf
3) Berechne für alle fehlenden Werte eine Vorhersage

- Jetzt wiederhole Schritt 1) bis 3) erneut und schätze die fehlenden Werte erneut - dieses Mal mit den bereits imputierten fehlenden Werten.
- Wiederhole dies, bis sich die imputierten Werte nicht mehr ändern.

AUFGABEN

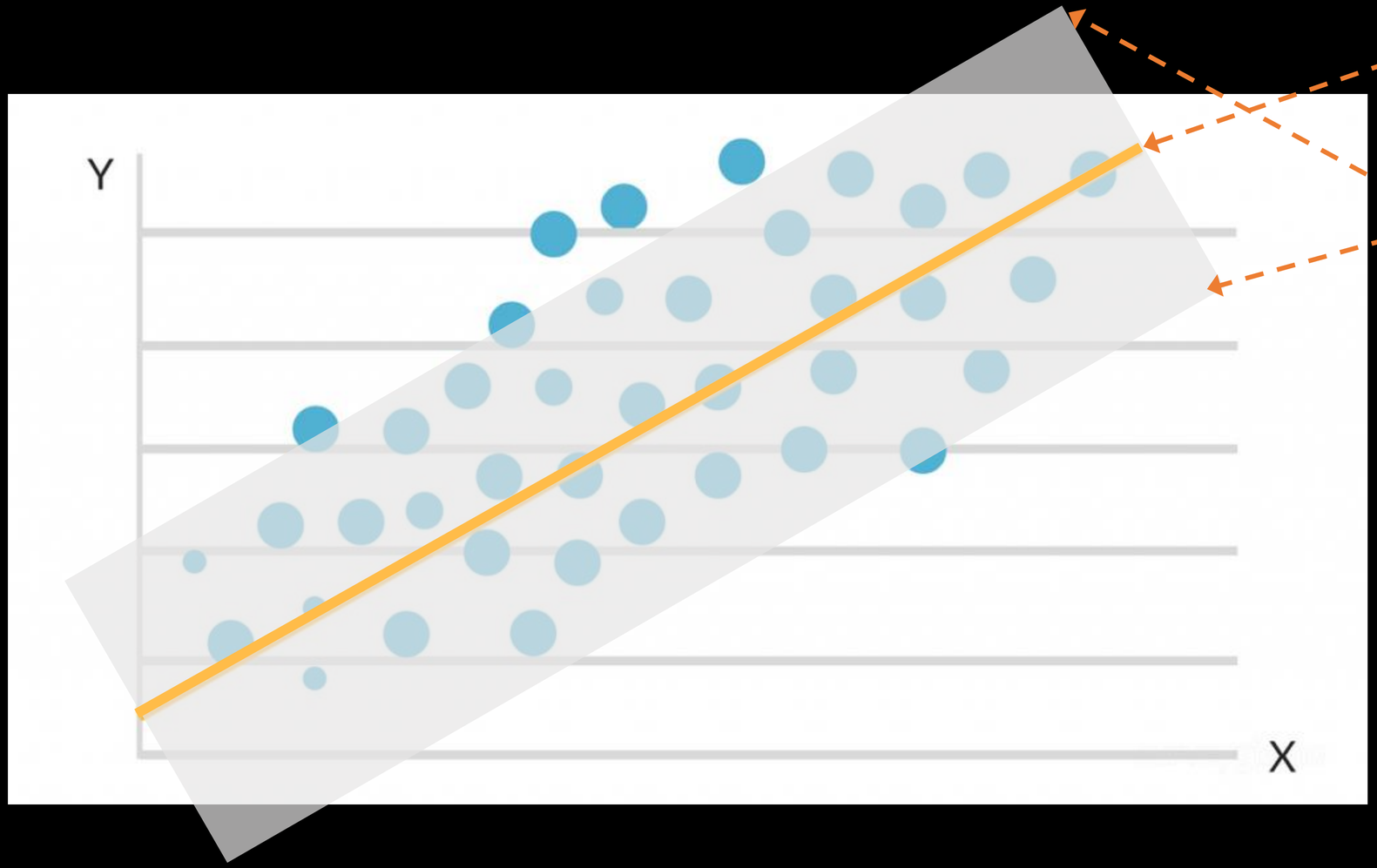
- **Wählt ein (bzw. verschiedene) Verfahren, um fehlenden Werte in Eurem Datensatz zu ersetzen.**

- **Schaut dieses Video (5 Minuten) zu Zeitreihenanalysen an.**

- **Teilt Euch die Aufgaben im Team gut auf:**
  Wer arbeitet an der Datenoptimierung, wer an der Modelloptimierung?

SUPPORT VECTOR MACHINE (SVM)

[SUPPORT VECTOR REGRESSION (SVR)]



Hyper plane

Boundary lines with soft margins

- Der Soft Margin Parameter (auch Cost Paramater) ist ein Regularisierungs-parameter, der die Flexibilität der Grenzlinien steuert bzw. wie stark Punkte außerhalb bestraft werden.

SUPPORT VECTOR MACHINES

Lernressourcen

- **Blog:** https://towardsdatascience.com/an-introduction-to-support-vector-regression-svr-a3ebc1672c2

- **Video:** https://www.youtube.com/watch?v=efR1C6CvhmE

OPTIMIERUNG EINER SVM IN R

R-Package „e1071“

`svm()`	**Optimierung eines SVM Modells**
`tune()`	**Optimierung mehrerer SVM Modelle mit gleichzeitiger Optimierung des C- und Gamma-Parameters**
`predict()`	**Vorhersage auf Basis eines optimierten Modells**

BEISPIELCODE IN R

```
---
title: "Support Vector Machine"
output: html_notebook
---


## Imports
## Splitting Training and Test Data
## Data Preparation
## Training the SVM

```{r}
# Optimization of an SVM with standard hyper parameters
# Typically NOT used; Instead, the function svm_tune() is used in order to also get a model with optimized hyper parameters
model_svm <- svm(price ~ bathrooms, train_dataset)
```

```{r}
# Optimization of various SVM using systematically varied hyper parameters (typically called 'grid search' approach) and cross validation
# the resulting object includes the optimal model in the element named 'best.model'
svm_tune <- tune(svm, price ~ bedrooms + bathrooms + sqft_living + zipcode, data=train_dataset,
                 ranges = list(epsilon = seq(0.2,1,0.1), cost = 2^(2:3)))
```


Checking the Prediction Quality
```

ZUSAMMENFASSUNG SVM

- Populärer, weil einfach zu optimierender Lernalgorithmus, der schnell gute Ergebnisse bringt.
- Geeignet zur Klassifikation und zur Regression.